Autonomous Cognitive System Theory, Phase 1 — an English overview

Companion note to the Japanese manuscript "自律認知システム理論：倫理的協調知性の数理的基盤（Phase 1：決定論的定式化）", by NF (a pen name). This note summarizes the manuscript. It is not a translation and adds no results. Where the note and the manuscript differ, the manuscript is authoritative. Labels such as `def:viab_core` refer to `\labels` in `paper/`.

Further reading

- Detailed English summaries: section-by-section summaries from the Abstract through §9.
- English glossary: the English terms used in the summaries.
- Japanese manuscript: the authoritative text.

1. What the paper is about

The paper gives a deterministic, discrete-time formal framework for describing autonomous agents — human or artificial — in a common structural vocabulary. Agent behaviour is often judged against social or cultural norms, and agents that live under different norms have little shared ground for understanding each other's actions. The paper instead relates an action to its *structural sources*: the feasibility requirements the agent is under, its internal state and internal model, the external environment, and the operation of its internal mechanisms. The aim is a shared description that does not presuppose any particular culture's judgments of good and bad.

The framework is **descriptive, not normative**. It does not take reward as a first principle, and its three principles are not objective functions for selecting actions (purposes formed inside an ACS are a separate, mechanism-level description: `def:purpose_formation`, §4). It does not predict actions, and it does not single out a unique cause of an action. Ethics enters only as motivation: the long-term programme (ECIT, Ethical Coordinative Intelligence Theory) treats ethical behaviour as constraints that keep a system feasible, rather than as commands. Phase 1 states this motivation informally and does not use it in any claim.

Scope of Phase 1. Deterministic dynamics, discrete time, compact spaces, no branching of identity, and mainly a single agent (organizations get a limited structural treatment). Probability is excluded by design and deferred to Phase 2.

2. The basic object (§2)

An ACS is a 7-tuple structure `Σ = (X, U, E, f, Π, M, Update)` that has a realization satisfying the constitutive requirements CR-1 and CR-2, relative to an external environment state space and under the Type Separation Convention (TSC) (`def:acs`). The components are:

- `X` internal states, `U` actions, and an external environment state space `E_ext = E^bg × O`, split into an exogenous background component and a component that responds to the agent's actions (`def:Eext_decomposition`). `E_ext` is a background type, not a tuple component.
- `E` is the set of **required environmental factors**. Each factor is a map `η : X × E_ext → {0,1}`; `η(x,e)=1` means the factor is satisfied in state `x` under environment `e` (`def:environment_space`). Constitutive requirement CR-1 (`req:CR1`) says what makes a factor "required": along some state-transition sequence, the factor, satisfied at a time when the realization is well-formed, must be able to remain unsatisfied at every later time; along every such sequence, the well-formed realization must be lost at every time from some finite time onward; for some state, some environment must satisfy all factors at once; and each factor's lasting non-satisfaction must also be realizable starting from a state at which some environment would satisfy it, so that satisfaction of the factor depends on the external environment state. This does not identify the environment as the cause of the non-satisfaction, since the internal state may also change.
- `f : X × U × E_ext → X` is the transition map (`def:state_transition`).

- Π is a persistence evaluation function and K = {x : Π(x) ≥ 0} the feasible set (def:Pi, def:K).
- M and Update are the internal model and its update operator (def:internal_model, def:update_operator).

Constitutive requirement CR-2 (req:CR2) asks that the realization have a basic function for each of the four primitive functions (sensing, discrimination, evaluation, action), that these functions be non-degenerate, and that it have the base functions supporting their activity. Having the functions is a matter of how the realization is constituted; CR-2 does not require them to be operating at every time (loss of operation is treated by functional termination, def:functional_termination).

Derived objects:

- E_req(x), the environment states in which all required factors hold (def:E_req).
- A(x,e) = {u : f(x,u,e) ∈ K}, the feasible action set (def:at). **Blockage** means A_t = ∅ (def:blockage_sec2).

The paper keeps several kinds of failure apart:

- blockage (no feasible action);
- loss of well-formed realization (def:wf_realization);
- **functional termination**: total functional stoppage that is not recovered (def:functional_termination);
- **collapse**: the loss of the organizational relations that make the agent the same structural individual, with no recovery as that individual within the framework (def:collapse).

If collapse has occurred by time t (Col_t = 1), a functional-termination state has occurred by that time (FT_t = 1) (prop:termination_order). None of the other conditions, on its own, is collapse.

3. Three principles as separate constraint axes (§3)

The three principles are *constraint axes*. They are not commands and not objectives. What has a truth value is a time-indexed satisfaction predicate, and only on the principle time domain T_P = {t : FT_t = 0} (def:principle_time_domain).

- **SM (self-maintenance):** SM_t holds iff x_t ∈ K (def:SM, prop:sm_K_equiv).
- **EU (environmental utilization):** a factor η is *one-step critical* at x if it can fail at x, whenever it fails no feasible action remains, and some environment satisfies it while leaving a feasible action (def:eu_critical_domain). EU_t holds iff every critical factor at x_t is satisfied by e_t (def:EU). If the time-indexed satisfaction predicate EU_t is false at a time of T_P, the agent is blocked at that time (prop:eu_blockage).
- **FR (feasible response):** at a time of T_P, FR_t is defined only when EU_t is true (def:fr_domain, prop:fr_domain_eu). There, FR_t holds iff A_t ≠ ∅ (def:FR). An undefined FR is not a false FR.

Together these give a dichotomy of blockage (prop:blockage_dichotomy): every blocked time is exactly one of (B1) a critical required factor is unmet, or (B2) all critical factors are met and there is still no feasible action.

A separate result says that satisfying the predicates does not by itself keep the agent feasible: choosing an action outside A_t leaves K at the next step (prop:selection_responsibility).

4. A viability kernel relative to required environments (§3)

The main set-valued object is the **E_req robust viability kernel** Viab_f(K) (def:viab_core). A set K' ⊆ K_req = K ∩ D_req is E_req-invariant if every x ∈ K' has a single action u such that f(x,u,e) ∈ K' for **all** e ∈ E_req(x). Viab_f(K) is the union of all such sets, and it is itself invariant, hence the largest one (prop:viab_core_max). Here D_req = {x ∈ X : E_req(x) ≠ ∅} is the required-environment-satisfying state domain (def:req_domain).

Relation to existing theory. The quantifier pattern — there exists an action, for all disturbances — is the same type as in discriminating and robust viability kernels (Cardaliaguet, Quincampoix and Saint-Pierre 1994; Aubin 1991). The paper does **not** claim that the definitions are identical: the state space, the range of disturbances, and the strategy and information assumptions can differ. What is specific to ACS is the disturbance range. It is the state-dependent set `E_req(x)`, generated by the required-factor maps `η` and constrained by CR-1, rather than a disturbance set fixed in advance. The paper also places its approach next to control barrier functions, safety shields and homeostatic reinforcement learning. It gives no method for synthesizing safe controllers.

Auxiliary results (assumption-free):

- Finite-stage robust sets `Viab_f^(ν)(K)` are nested and contain the kernel (`prop:viab_stage_nested`, `prop:viab_stage_core`).
- If `X` is finite, the stages stabilize after at most `|K_req|` steps and their intersection equals the kernel. A state of `K_req` then has an infinite robust horizon iff it lies in the kernel (`prop:viab_stage_finite`). For infinite `X` this equality is not claimed.
- Compared with the all-environment robust kernel `Viab^all` and the optimistic kernel `Viab^any` (`def:kernel_all_any`), `Viab_f(K) ⊆ Viab^any`. Also, `Viab^all ⊆ Viab_f(K)` iff `Viab^all ⊆ D_req` (`prop:kernel_standard_comparison`). Without that condition the inclusion can fail.
- If the environment is a required one and the agent is blocked, the state is outside the kernel (`prop:blockage_viability_kernel`, §7).

Conditional kernel maintenance (`prop:kernel_maintenance`, no assumption). Suppose `x_T ∈ Viab_f(K)` and, for every `s ≥ T`, the realized environment satisfies all required factors (`e_s ∈ E_req(x_s)`) and the response is kernel-maintaining (`u_s ∈ U^Viab(x_s)`, `def:kernel_response`). Then for every `s ≥ T` the state stays in `Viab_f(K)` and `A_s ≠ ∅`. This is a conditional feasibility-preservation result. It says nothing about whether the premises hold, and it is not a claim of survival or of the acquisition of such responses by an internal mechanism.

Response-diversity degeneration (`prop:sd_robust_nonresponse`, main proposition, no strong assumption): suppose that from some time on the realized environment is a required one whenever the agent is in `K`. Suppose also that, whenever the agent is in `K` and an action is feasible, the response diversity `μ_eff` equals 1 (all feasible actions retain feasibility over exactly the same required-satisfying environment states) and there is a weak point of a feasible response. Then, at every time from then on, the state is outside the kernel, and at times in `K` no response is robust (`U^rob(x) = ∅`). Under the additional strong assumption `ass:theta_exterior_blockage`, blockage follows in finite time (`prop:sd_necessity`, corollary). The paper keeps the two statements separate.

5. Mechanism level, dynamics, relations, collapse, organizations, limits (§4–§9)

- **§4 Standard mechanism form.** The internal model structure `M = (M1, M2, M3)` of the 7-tuple is treated as TIPA (Triadic Information-Processing Architecture), with the information-integration mechanism M1, the adaptive-evaluation mechanism M2 and the response-action mechanism M3 (`def:acs`). PSL (Physical Substrate Layer) connects TIPA with input, output, substrate maintenance and local protective responses on the implementation side. ISL (Implementation Substrate Layer) organizes the implementation carriers that can carry sensing, action, information retention and the like. These are *standard functional forms* (classification B); the paper does not require their elements to exist as distinct, one-to-one physical parts or computational modules. The principle level (SM, EU, FR) is not identified one-to-one with any mechanism, and mechanism outputs alone do not determine the truth of the principle predicates (`rem:principle_level_locus`).
- **§5 Time-to-time connection.** One-step operation records with well-formedness, continuation and causality properties. Descriptive read-outs such as bias strength, effective time horizon, effective development freedom and health indices are classification C. The paper assumes no universal learning rule, convergence or monotonicity.
- **§6 Relations.** A value-neutral, non-causal descriptor of one-directional relations to external objects, with a bidirectional composition and 18 representative category labels (15 possible values of the category component of a unidirectional descriptor, and 3 auxiliary

labels for bidirectional relations).

- **§7 Collapse and paths.** Descriptive state bands, an admissible set of external actions and an external-recovery predicate, path sets reaching blockage and reaching collapse, and the corresponding reaching times. The two path sets are not assumed to be nested. "Inflow tendencies" are descriptive connections, not implications.
- **§8 Organizational ACS.** An organization made of several participants is treated as an organizational ACS when the organization itself realizes a well-formed ACS-shaped structure `Σ_org`. Participation of individual ACSs is not required; the number of participating ACSs is recorded and only separates organizational ACSs with and without participating individual ACSs. Individual collapse and organizational collapse are kept distinct. A society is treated as the setting to which ACSs and other agents belong and in which they act.
- **§9 Limits and outlook.** Eighteen stated limits (L1–L18), items delegated to Phase 1.α, eleven open problems, the outlook for Phases 2 and 3, and the conclusion. The open problems are research problems; the paper does not claim that any of them is solvable.

6. Assumptions and claim strength

The paper states 21 assumptions, each labelled with its status (A = axiomatic, S = structural, O = observational) and its strength (weak, medium, strong). Five are strong: four in §2 (`ass:no_branching`, `ass:compact_spaces`, `ass:f_decomposition`, `ass:M_compact`) and `ass:theta_exterior_blockage` in §3. The assumption table in §2 lists where each assumption is mainly used, and `docs/dependency_table.md` gives a mechanically extracted index of the references in propositions and their proofs, with its limits stated. The paper contains no theorems. Its results are 48 propositions and 3 lemmas, several of which are definitional or standard in viability theory.

7. What the paper does not claim

- No empirical claim about real agents, and no general law of action selection.
- No claim that satisfying the three principles, choosing kernel-maintaining responses, or having the TIPA, PSL or ISL mechanism makes an agent survive longer.
- No claim that `Viab_f(K)` is superior to standard kernels. It is the largest `E_req`-invariant set (`prop:viab_core_max`), defined relative to a state-dependent disturbance set.
- No normative or ethical verdicts.

8. Companion toy models

The `toy/` package evaluates the paper's definitions on three small finite models: two lattice agents with a non-ACS rescuer, "MinACS" with |X| = 72, and a furnace keeper with |X| = 60. MinACS and the furnace keeper are computed by exhaustive enumeration. In the lattice model, membership in the kernel is decided by certificates (a constructed invariant set), and non-membership by a finite robust horizon. Its mechanisms see only their own observations; the true `f`, `K` and kernel are used only by a separate judging layer. Experiments were preregistered: hypotheses, judging rules and seed splits were frozen together with source hashes before evaluation. The results illustrate existence and the conceptual distinctions. Hypotheses that would have favoured the framework were **not supported** in these models: that choosing kernel-maintaining responses survives at least as long as the TIPA mechanism, that following the three principles extends survival, and that having the full set of mechanism functions extends survival. The toys are not evidence for the paper's propositions or for real-world applicability. Independent AI-assisted code checks have been performed. No independent human third-party verification of the code or mathematical arguments has yet been conducted.

Phases

- **Phase 1** (this paper): deterministic formulation.
- **Phase 1.α**: rules, thresholds and operational criteria made concrete, within the same types.
- **Phase 2**: stochastic and continuous-time extensions, causal relational dynamics, branching identity, quantitative organizational and social dynamics.
- **Phase 3**: the formal connection to ECIT.